XSUAA Implementation Guide

SAP CAP (Node.js) • EmployeeManagement project • 22 October 2026

This document has two parts. **Part A** records what was changed in the EmployeeManagement project to add XSUAA authentication and role-based authorization. **Part B** is a reusable checklist for adding XSUAA to the next CAP project, in the order the steps should be done.

How the pieces fit together

```
Browser
  |  1. user opens the app
  v
Managed App Router (HTML5 Apps / Work Zone)
  |  2. no session -> redirects to XSUAA login (redirect-uris must allow this URL)
  |  3. after login, user has a JWT containing the scopes of their role collections
  |  4. xs-app.json route /odata/*  -> destination "EmployeeManagement-srv-api"
  |     (HTML5.ForwardAuthToken: true sends the JWT along)
  v
CAP service (srv)  - auth: "xsuaa", checks JWT with @sap/xssec
  |  5. @requires / @restrict in CDS compare the user's roles (Viewer / Admin)
  v
HANA
```

Roles are defined in CDS, declared to XSUAA as scopes and role templates in `xs-security.json`, grouped into role collections, and assigned to users in the BTP cockpit.

Role design used

Role (CDS)	Role collection (BTP)	Can do
Viewer	EmployeeMgmt_Viewer	Read Employees and Departments
Admin	EmployeeMgmt_Admin	Create, read, update, delete Employees and Departments (Admin also includes the Viewer scope)
(no role)	–	Logged in but gets HTTP 403 on every entity
(not logged in)	–	HTTP 401

Part A – Changes made in this project

What already existed before the change

- `package.json`: `"[production]": { "auth": "xsuaa" }` and the `@sap/xssec` dependency.
- `mta.yaml`: an `xsuaa` service instance `EmployeeManagement-auth` (plan `application`) bound to the srv module.
- `xs-app.json`: the catch-all UI route already used `authenticationType: xsuaa`.

What was missing: `xs-security.json` was empty (no scopes/roles), the service had no authorization annotations, there were no mock users for local testing, and no route forwarded the user's token from the UI to the backend.

1. srv/employee-service.cds – authorization annotations

`requires: authenticated-user` blocks anonymous access; `@restrict` grants per role.

```
service EmployeeService @(
  path    : '/odata/v4/employee',
  requires: 'authenticated-user'
) {
  @restrict: [
    { grant: 'READ', to: 'Viewer' },
    { grant: '*',    to: 'Admin' }
  ]
  entity Employees   as projection on db.Employees;

  @restrict: [
    { grant: 'READ', to: 'Viewer' },
    { grant: '*',    to: 'Admin' }
  ]
  entity Departments as projection on db.Departments;
}
```

2. xs-security.json – scopes, role templates, role collections

Scope names must match the role names in CDS (Viewer, Admin). Role collections are created automatically on deploy, so they only need to be assigned to users afterwards.

```
{
  "xsappname": "EmployeeManagement",
  "tenant-mode": "dedicated",
  "scopes": [
    { "name": "$XSAPPNAME.Viewer", "description": "Read employees and departments" },
    { "name": "$XSAPPNAME.Admin",  "description": "Create, update and delete ..." }
  ],
  "role-templates": [
    { "name": "Viewer", "scope-references": ["$XSAPPNAME.Viewer"] },
    { "name": "Admin",  "scope-references": ["$XSAPPNAME.Admin", "$XSAPPNAME.Viewer"] }
  ],
  "role-collections": [
    { "name": "EmployeeMgmt_Viewer", "role-template-references": ["$XSAPPNAME.Viewer"] },
    { "name": "EmployeeMgmt_Admin",  "role-template-references": ["$XSAPPNAME.Admin"] }
  ]
}
```

The `xsappname` in mta.yaml (`EmployeeManagement-${org}-${space}`) overrides the one in this file.

3. package.json – mock users for local development

```
"cds": {
  "requires": {
    "[production]": { "auth": "xsuaa" },
    "[development]": {
      "auth": {
        "kind": "mocked",
        "users": {
          "admin":  { "password": "admin",  "roles": ["Admin", "Viewer"] },
          "viewer": { "password": "viewer", "roles": ["Viewer"] }
        }
      }
    }
  }
}
```

4. mta.yaml – deployment descriptor

- **XSUAA resource:** added `redirect-uris` (`https://*.hana.ondemand.com/**`, `https://*.applicationstudio.cloud.sap/**`) so the login redirect is accepted.
- **Destination service resource:** `HTML5Runtime_enabled` set to `true`, now `requires: srv-api`, and a new destination `EmployeeManagement-srv-api` pointing at the CAP service URL with `HTML5.ForwardAuthToken: true`.
- **App deployer module:** now only uploads the UI zip to the HTML5 repository (requires `EmployeeManagement-html5-repo-host` as content target).
- **New module** `EmployeeManagement-destination-content`**:** creates the destinations the managed app router needs – one to the HTML5 repo host and one to XSUAA (`OAuth2UserTokenExchange`), both tagged `sap.cloud.service: employeemanagement`.

```
- name: EmployeeManagement-srv-api          # in destination-service init_data
  Authentication: NoAuthentication
  ProxyType: Internet
  Type: HTTP
  URL: ~{srv-api/srv-url}
  HTML5.DynamicDestination: true
  HTML5.ForwardAuthToken: true
```

5. app/employeemgntui/xs-app.json – backend route

Added before the catch-all route (routes are matched top to bottom):

```
{
  "source": "^/odata/(.*)$",
  "target": "/odata/$1",
  "destination": "EmployeeManagement-srv-api",
  "authenticationType": "xsuaa",
  "csrfProtection": false
}
```

6. app/employeemgntui/webapp/manifest.json

- Data source URI changed from `/odata/v4/employee/` to the relative `odata/v4/employee/`, so requests go through the app router's route instead of the domain root.
- Added `"sap.cloud": { "public": true, "service": "employeemanagement" }`. The service name must equal `sap.cloud.service` in mta.yaml.

```
"sap.cloud": {
  "public": true,
  "service": "employeemanagement"
}
```

Why the sap.cloud block is needed

The project is deployed to the **managed app router** (deploy_mode: html5-repo). Without this block the UI is uploaded but cannot be opened with an XSUAA login.

`service` is the name that links the UI to its XSUAA instance:

- The UI is stored in the HTML5 Application Repository under the name employeemanagement.
- The `EmployeeManagement-destination-content` module in mta.yaml creates two destinations with the same name (`sap.cloud.service: employeemanagement`): `..._html_repo_host` (where the UI files are) and `..._auth` (which XSUAA instance to log in with – EmployeeManagement-auth with the Viewer/Admin roles).
- When a user opens the app, the managed app router reads this name, finds the matching destinations and logs the user in with the right XSUAA, so the token contains the correct roles.
- The name is also part of the app URL, e.g. `https://<subaccount>.launchpad.cfapps.../employeemanagement.employeemgntui/`.

`public: true` lets the app be used outside its own service, e.g. as a tile in SAP Build Work Zone. With false, only apps under the same service name can reach it.

If it is missing or the names differ: the app does not appear under HTML5 Applications in the cockpit, and opening it fails or logs in against the wrong XSUAA. The value in manifest.json and both sap.cloud.service values in mta.yaml must be identical. Locally (cds watch) the block has no effect.

7. app/employeemgntui/ui5.yaml – local proxy

Because the URI is now relative, local runs need `/odata` proxied to the CAP server:

```
- name: fiori-tools-proxy
  configuration:
    backend:
      - path: /odata
        url: http://localhost:4004
```

Verification done

Local test with mocked authentication (in-memory SQLite) against /odata/v4/employee/Departments:

Request	Expected	Result
GET, not logged in	401	401
GET as viewer	200	200
POST as viewer	403	403
POST as admin	201	201

mta.yaml parses and the CDS model compiles. **Not yet verified:** the deployment to BTP (login redirect, destinations, token forwarding) – run the steps in B7–B9 after deploying.

Part B – Checklist for the next project

Follow the steps in order. Each step lists what to do and how to check it worked.

B1. Decide the roles

Write down who uses the app and what each group may do (e.g. Viewer = read, Admin = everything, Manager = only own department). Keep role names short, PascalCase, no spaces.

B2. Add XSUAA to the project

Run the CAP generator. It adds the xsuaa dependency, the production auth config, the mta.yaml resource and a starter xs-security.json:

```
npm add @sap/xssec
cds add xsuaa
cds add mta          # if mta.yaml does not exist yet
```

B3. Annotate the services in CDS

Put `@requires` on the service (at least `authenticated-user`) and `@restrict` on entities or actions. For row-level rules use a where clause, e.g. `{ grant: 'READ', to: 'Manager', where: 'department_ID = $user.dept' }` (needs an attribute in xs-security.json).

```
service MyService @(requires: 'authenticated-user') {
  @restrict: [ { grant: 'READ', to: 'Viewer' }, { grant: '*', to: 'Admin' } ]
  entity Things as projection on db.Things;
}
```

B4. Write xs-security.json

One scope per CDS role ($XSAPPNAME.RoleName), one role template per role, and role collections so roles can be assigned without manual cockpit setup. Add redirect-uris (in xs-security.json or in the mta.yaml resource config). If you add roles later, rerun cds add xsuaa or edit by hand – scope names must match CDS role names exactly.

B5. Add mock users and test locally

Add `[development]` mocked users to package.json (one per role), run `cds watch`, and check each role with curl or the browser login prompt:

```
curl -i localhost:4004/odata/v4/my/Things                # 401
curl -i -u viewer:viewer localhost:4004/odata/v4/my/Things # 200
curl -i -u viewer:viewer -X POST -H 'Content-Type: application/json' \
     -d '{}' localhost:4004/odata/v4/my/Things            # 403
```

B6. Connect UI → backend through the app router

mta.yaml: destination to the srv URL with HTML5.ForwardAuthToken: true; destination-content module with HTML5-repo and XSUAA destinations tagged with the same sap.cloud.service.

xs-app.json: route ^/odata/(.*)$ → that destination, authenticationType xsuaa, placed above the catch-all.

manifest.json: relative data source URI, and a sap.cloud block (public: true, service: <name>) – the managed app router uses this name to find the app's XSUAA and HTML5-repo destinations, so it must match mta.yaml exactly.

ui5.yaml: fiori-tools-proxy backend /odata → http://localhost:4004 so local runs still work.

B7. Build and deploy

```
npm run build        # mbt build
npm run deploy       # cf deploy mta_archives/archive.mtar
```

B8. Assign role collections

BTP Cockpit → subaccount → Security → Users (or Role Collections) → add the app's role collections to your user / user group. Log out and in again so the new token contains the scopes.

B9. Test in the cloud

Open the app from HTML5 Applications (or Work Zone). Check: login page appears, data loads, read-only user cannot save. To inspect your token's scopes, decode the JWT or call an endpoint that returns `req.user`.

Troubleshooting

Symptom	Likely cause	Fix
401 from the service after login	Token not forwarded to srv	Destination needs HTML5.ForwardAuthToken: true; route must use that destination
403 for every request	User has no role collection, or old token	Assign role collection, log out / in
403 although role is assigned	Scope name ≠ CDS role name	Make $XSAPPNAME.X match the to: 'X' in @restrict
Login error 'redirect_uri not registered'	redirect-uris missing or too narrow	Add the app router URL pattern to oauth2-configuration.redirect-uris
App not listed in HTML5 Applications	sap.cloud.service mismatch	Same value in manifest.json and both destination-content destinations
404 on /odata locally	Relative URI without local proxy	Add fiori-tools-proxy backend for /odata
Deploy error on xs-security.json changes	Removed/renamed scopes on existing instance	Update is usually fine; renames may need cf update-service or recreating the instance

Useful commands

```
cds add xsuaa                # scaffold XSUAA config
cds watch                    # local run with mocked users
cds watch --profile hybrid   # local run against real XSUAA (after cds bind)
cds bind -2 EmployeeManagement-auth   # bind local app to the cloud XSUAA instance
cf service-key EmployeeManagement-auth EmployeeManagement-auth-key  # inspect credentials
```